Smart Gate v3: Unified Build Log

Purpose

This log records how the team got from the inherited v0.2 system to the working v3 bench: what was done in each phase, what went wrong, and how it was fixed. It was compiled from each member's working notes. Use it to understand why the system is set up the way it is; for the formal record of each change, see the Change Register.

Timeline summary

Phase	Dates	Lead	Outcome
1. Planning and procurement	11 Aug – 12 Sep	Antus, Thea	Plan, schedule and component inventory in place
2. Motor-driver prototypes	8 – 26 Sep	Janvy, Thea, Krishna	L298N logic validated manually and on a Raspberry Pi
3. Orin set-up and environment	22 Sep – 14 Oct	Martin, Krishna	Orin flashed; Docker image built
4. Model and TensorRT	14 – 21 Oct	Martin, Krishna	Engine rebuilt and detection running
5. Camera bring-up	29 Sep – 3 Oct	Antus	IMX219 streaming on the Orin
6. GPIO and motor pins	16 – 24 Oct	Thea	Pin fix found; boot script written
7. Stepper and Hall integration	15 – 28 Oct	Thea, Martin	Stepper stops at both end stops
8. Detection tuning and testing	21 – 30 Oct	Team	Threshold set to 0.6; batch test recorded
9. Test bench	15 Sep – 24 Oct	Martin	Open modular bench printed and assembled
10. Web dashboard	20 – 30 Oct	Martin, Krishna	Dashboard drives the gate
11. Integration and handover	27 Oct – 5 Nov	Team	Full chain demonstrated to OpenSI
12. Repository and documentation	31 Oct – 1 Nov	Team	Code pushed to GitHub; README, change register and these docs written

Phase 1: Planning and procurement

- Built the WBS, critical-path schedule and Gantt chart; MVP critical path ran from procurement through flashing, GPIO configuration, wiring and testing to owner review.
- Compiled the component inventory (Jetson Orin dev kit, PIR, camera, CSI cables, L298N, jumper wires, microSD cards, Hall sensors, resistors, LEDs). See `docs/inventory/`.

Phase 2: Motor-driver prototypes

- Stage 1: wired the L298N on a breadboard with a 9 V pack and LEDs; measured both directions and the disabled state.
- **Problem:** inconsistent output on ENB. **Fix:** replaced the driver.
- Stage 2: drove IN3, IN4 and ENB from a Raspberry Pi 3B+ in Python because the Orin had not arrived. Same three states confirmed.

Phase 3: Orin set-up and environment

- Flashed the Orin Nano (JetPack 7.2) and built the `smartgate` Docker image (CUDA 13.2, OpenCV with GStreamer), with GPU, camera and GPIO access.
- Added the `sgate` alias to `~/.bashrc`; it mounts `~/SmartGate` as `/app` so code edits need no rebuild.
- **Note:** the image takes about two hours to rebuild, so `docker system prune -a` must not be run.

Phase 4: Model and TensorRT

- **Problem:** the inherited TensorRT engine would not run on the Orin, because engines only run on the device and TensorRT version they were built for.
- **Fix:** rebuilt the engine (TensorRT 10.16.2) from the inherited YOLOv5 weights with `setup/convert_models.sh`; weights and class list unchanged.
- Known warning: "engine built on a different device" may still appear. Its cause was not confirmed; detection works normally, so it is treated as harmless.

Phase 5: Camera bring-up

- Configured the IMX219 overlay with `jetson-io` and rebooted.
- **Problem:** no `/dev/video0`; `dmesg` showed the driver loaded but no reply on the camera bus (error -121).
- **Fix:** set the IMX219 overlay in `jetson-io` to match the CSI port in use, reseated the 22-to-15-pin ribbon cable and corrected its orientation at the Jetson end. Camera now streams.

Phase 6: GPIO and motor pins

- **Problem:** the motor did not move. The Orin's header pins boot as inputs.
- **Fix:** identified four register writes for pins 16, 18, 29 and 31. Correct readback is 5, 5, 8, 8; 55, 55, 58, 58 means the fix did not apply.
- The settings reset on every restart, so a boot script (`setup/sgate_boot.sh`) now applies them at start-up.
- **Problem (24 Oct):** re-running the inherited `setup/systemd_service_setup.sh` overwrote the working service, so the pins went back to defaults and the gate stopped. **Fix:** added the `sgate_boot.sh` step and the live-code mount to the setup script itself, so it always generates the working service. The system now starts automatically at every boot.

Phase 7: Stepper and Hall integration

- Scope change on 15 October: Alex (OpenSI) suggested a stepper motor with Hall end stops instead of LEDs; the team agreed.
- Merged stepper control into the existing `io_control.py` and `door_control.py`. The original DC-motor versions are kept as `io_control_dc.py` and `door_control_dc.py`, and an earlier stepper-only version as `io_control_old.py` and `door_control_old.py` (which is only the stepper controller).
- **Problem:** Hall sensors unreliable at 3.3 V. **Fix:** powered them at 5 V; output stays at a safe 3.34 V.
- Stepper runs at 0.25 s per step, 48 steps per revolution, and stops at the open (pin 11) and close (pin 13) sensors. Travel limit `MAX_STEPS` (240) is a placeholder.

Phase 8: Detection tuning and testing

- **Problem:** at the inherited 0.5 threshold, a person was read as Fallow Deer (0.51–0.57) and closed the gate.
- **Fix:** raised the threshold to 0.6, agreed with OpenSI. Model-only misreads fell to 5 of 120 frames; no false close in integrated runs; fox still detected at 0.69.

- Ran the batch test: 75 of 80 animal images detected (94%). Full data in `docs/testing/`.

Phase 9: Test bench

- Version 1 (enclosed case) hid parts and wiring, so it was replaced by Version 2, an open two-deck modular bench.
- **Problems:** the hole grid printed larger than intended and joints were fragile. **Fixes:** Velcro mounting; lower-deck legs hot-glued. Details in `docs/cad/`.

Phase 10: Web dashboard

- Built the web dashboard in `src/web-app` (Node.js and PostgreSQL in Docker, port 8000): a Dashboard page (`stats.html`) with detection charts and a detection log, and a Live Stream page (`gates.html`) with the live feed, Open/Close/Stop and Auto/Manual controls, sensor readings and gate analytics. Reachable from other devices on the local network, with GitHub login and admin roles.
- Added live status updates, an address panel and a `smartgate.local` name for the Jetson (set on the Nano, not in the repo).
- Tested driving the stepper from the dashboard on the bench.
- **Problem:** the Nano gets a new, random IP address on every boot, so the web app's `JETSON_LIVE_URL` setting goes out of date. **Workaround:** the dashboard is started by hand after each boot, once the current address is set in `src/web-app/.env`. The gate service itself still starts automatically.

Phase 11: Integration and handover

- Demonstrated the full chain to OpenSI: motion, detection, decision, stepper to the Hall stop, and manual control.
- Deliverables verbally accepted; written confirmation requested. Bench to be handed over after the 5 November presentation.

Phase 12: Repository and documentation

- **Problem (31 Oct):** pushing from the Nano failed because old history held `yolov5/`, a Python venv (`.convert_venv`) and a 308 MB torch wheel over GitHub's 100 MB limit, and the Nano ran out of memory packing them.
- **Fix:** pushed a single clean commit (orphan branch replacing `main`) with a `.gitignore` excluding `yolov5/`, `*.pt`, `core`, venvs, `setup/torchvision/`, `__pycache__/` and `*.whl`.
- Rewrote the README and wrote the change register, wiring, testing, bench, build-log and operating-guide documents.